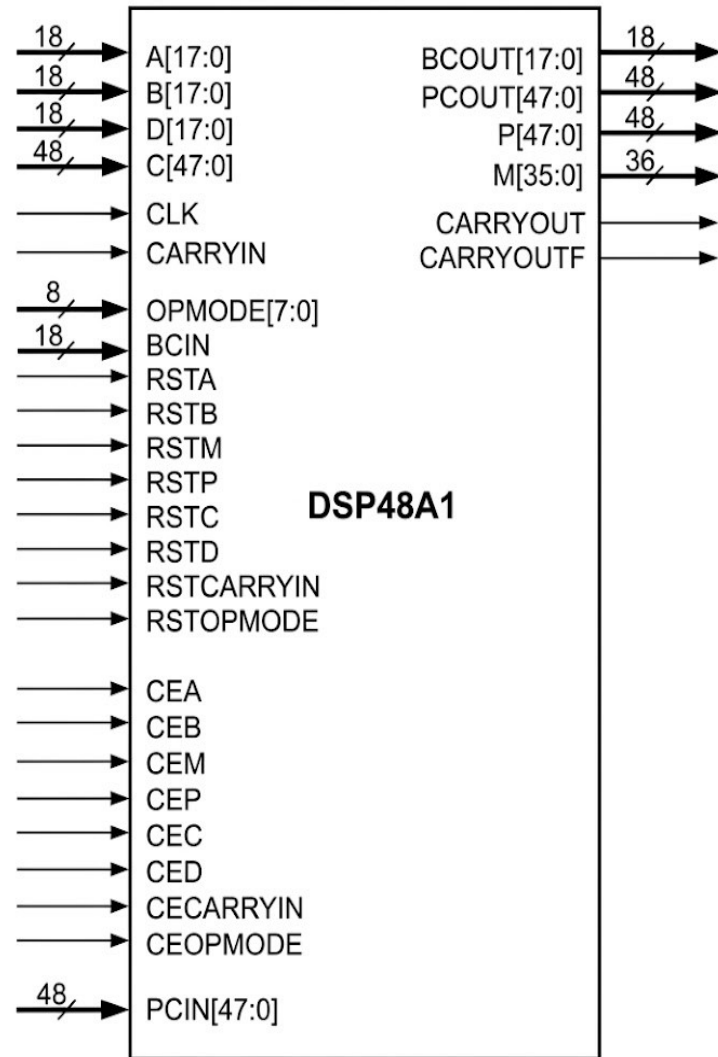


DSP48A1

BY : Ahmed Goda Sharawy



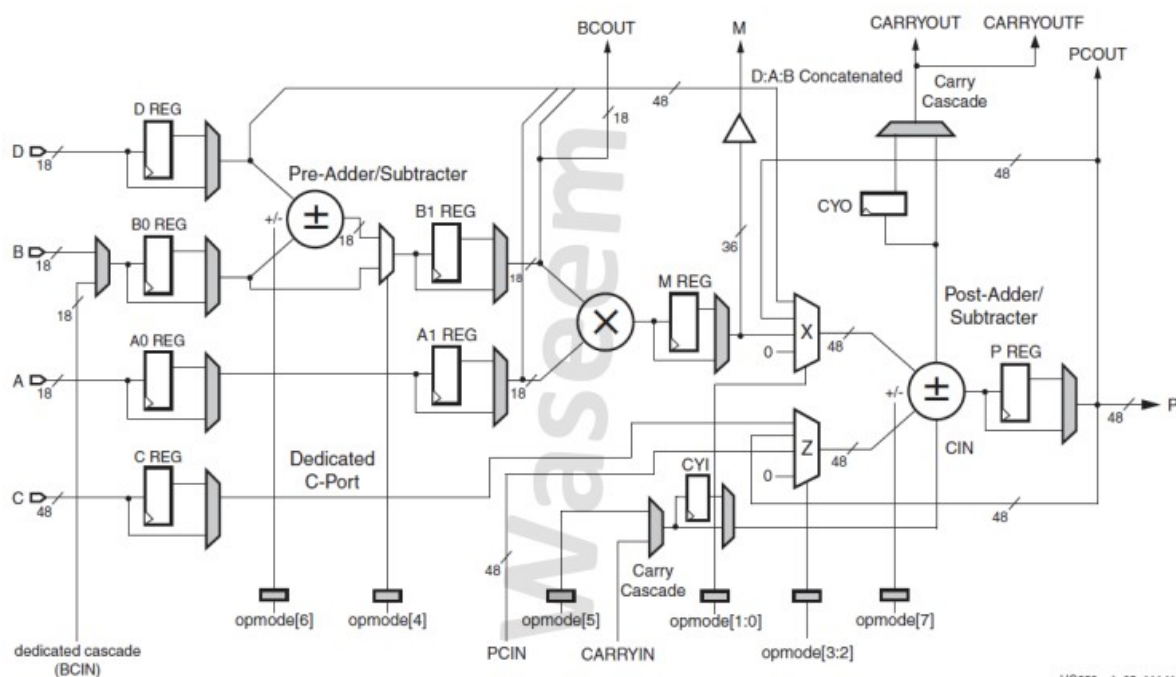
1. Project Overview

The **DSP48A1** is a dedicated Digital Signal Processing block available in the **Xilinx Spartan-6 FPGA family**. It provides high-speed arithmetic operations such as multiplication, addition, subtraction, and accumulation. In this project, the DSP48A1 block was implemented using **Verilog RTL** and verified through simulation to evaluate its functionality and different operating modes.

2. Project Modules

2.1 DSP48A1 Module

The design is divided into four stages to simplify the implementation and allow each stage to be developed and verified independently.



RTL Code

verilog > DSP48A1.v > DSP48A1 > [] C

```

1  module DSP48A1 #(
2      // PARAMETERS
3      parameter A0REG = 0,
4      parameter A1REG = 1,
5      parameter B0REG = 0,
6      parameter B1REG = 1,
7      parameter CREG = 1,
8      parameter DREG = 1,
9      parameter MREG = 1,
10     parameter PREG = 1,
11     parameter CARRYINREG = 1,
12     parameter CARRYOUTREG = 1,
13     parameter OPMODEREG = 1,
14     parameter CARRYINSEL = "OPMODE5",
15     parameter B_INPUT = "DIRECT",
16     parameter RSSTYPE = "SYNC"
17 ) (
18     // INPUTS
19     input wire [17:0] A,B,D,
20     input wire [47:0] C,
21     input wire CLK,
22     input wire CARRYIN,
23     input wire [7:0] OPMODE,
24     input wire [17:0] BCIN,
25     input wire RSTA,
26     input wire RSTB,
27     input wire RSTM,
28     input wire RSTP,
29     input wire RSTC,
30     input wire RSTD,
31     input wire RSTCARRYIN,
32     input wire RSTOPMODE,
33     input wire CEA,
34     input wire CEB,
35     input wire CEM,
36     input wire CEP,
37     input wire CEC,
38     input wire CED,
39     input wire CECARRYIN,
40     input wire CEOPMODE,
41     input wire [47:0] PCIN,
42     // OUTPUTS
43     output wire [17:0] BCOUT,
44     output wire [47:0] PCOUT,
45     output wire [47:0] P,
46     output wire [35:0] M,
47     output wire CARRYOUT,
48     output wire CARRYOUTF
49 );

```

```

50 // stage 1
51 wire [17:0] A_r,B_r,D_r;
52 wire [47:0] C_r;
53 wire [17:0] B_out;
54 wire [7:0] opmode_r;
55 // stage 2
56 reg [17:0] out1_ADD_or_SUB;
57 wire [17:0] out_opmode4;
58 wire [17:0] out_A1REG;
59 wire [17:0] out_B1REG;
60 wire [35:0] mul_out;
61 wire [47:0] conc;
62 //stage 3
63 wire [35:0] mul_out_reg;
64 reg [47:0] X;
65 reg [47:0] Z;
66 wire carry_cascade;
67 wire CIN;
68 // stage 4
69 reg [48:0] out2_ADD_or_SUB; // 1 bit for CIN
70 wire CY0; // COUT

```

```

74 //----- S T A G E 1 -----
75 // (1) instantiate DREG
76 REGISTER #(.WIDTH(18),
77             .RSSTYPE(RSSTYPE),
78             .REG(DREG))
79 u0 (.in(D),
80     .CLK(CLK),
81     .rst(RSTD),
82     .en(CED),
83     .out(D_r));
84 // (2) instantiate B0REG
85 REGISTER #(.WIDTH(18),
86             .RSSTYPE(RSSTYPE),
87             .REG(B0REG))
88 u1 (.in(B_out),
89     .CLK(CLK),
90     .rst(RSTB),
91     .en(CEB),
92     .out(B_r));
93 // B_INPUT
94 generate
95     if (B_INPUT == "DIRECT")begin
96         assign B_out = B;
97     end
98     else if (B_INPUT == "CASCADE")begin
99         assign B_out = BCIN;
100     end
101     else
102         assign B_out = 18'd0;
103 endgenerate
104 // (3) instantiate A1REG
105 REGISTER #(.WIDTH(18),
106             .RSSTYPE(RSSTYPE),
107             .REG(A0REG))
108 u2 (.in(A),
109     .CLK(CLK),
110     .rst(RSTA),
111     .en(CEA),
112     .out(A_r));

```

```

113 // (4) instantiate CREG
114 REGISTER #(
115     .RSSTYPE(RSSTYPE),
116     .REG(CREG))
117 u3 (.in(C),
118     .CLK(CLK),
119     .rst(RSTC),
120     .en(CEC),
121     .out(C_r));
122 // (5) instantiate OPMODEREG
123 REGISTER #(.WIDTH(8),
124             .RSSTYPE(RSSTYPE),
125             .REG(OPMODEREG))
126 u5 (.in(OPMODE),
127     .CLK(CLK),
128     .rst(RSTOPMODE),
129     .en(CEOPMODE),
130     .out(opmode_r));
131

```

```

132 //----- S T A G E 2 -----
133
134 // ADDER OR SUBTRACT 1 OPMODE[6]
135 always @(*) begin
136     if (OPMODE[6] == 0)
137         out1_ADD_or_SUB = D_r + B_r;
138     else
139         out1_ADD_or_SUB = D_r - B_r;
140 end
141 assign out_opmode4 = (OPMODE[4] == 0)? B_r : out1_ADD_or_SUB ;
142 // (6) instantiate B1REG
143 REGISTER #(.WIDTH(18),
144             .RSSTYPE(RSSTYPE),
145             .REG(B1REG))
146     u6 (.in(out_opmode4),
147         .CLK(CLK),
148         .rst(RSTB),
149         .en(CEB),
150         .out(out_B1REG));
151 // (7) instantiate A1REG
152 REGISTER #(.WIDTH(18),
153             .RSSTYPE(RSSTYPE),
154             .REG(A1REG))
155     u7 (.in(A_r),
156         .CLK(CLK),
157         .rst(RSTA),
158         .en(CEA),
159         .out(out_A1REG));
160
161 // MULTIPLICATION
162 assign mul_out = out_B1REG * out_A1REG;
163 // BCOU
164 assign BCOU = out_B1REG;
165 // CONCATINATION FOR WIRES D:A:B
166 assign conc = {D_r[11:0] , out_A1REG ,out_B1REG};
167

```

```

168 //----- S T A G E 3 -----
169 // (8) instantiate MREG
170 REGISTER #(.WIDTH(36),
171             .RSSTYPE(RSSTYPE),
172             .REG(MREG))
173     u8 (.in(mul_out),
174         .CLK(CLK),
175         .rst(RSTM),
176         .en(CEM),
177         .out(mul_out_reg));
178
179 // BUFFER M
180 assign M = mul_out_reg;
181
182 // MUX X
183 always @(*) begin
184     case (OPMODE[1:0])
185     2'b01: X = mul_out_reg;
186     2'b10: X = P;
187     2'b11: X = conc;
188     default: X = 48'd0;
189     endcase
190 end
191
192 // MUX Z
193 always @(*) begin
194     case (OPMODE[3:2])
195     2'b01: Z = PCIN;
196     2'b10: Z = P;
197     2'b11: Z = C_r;
198     default: Z = 48'd0;
199     endcase
200 end
201

```

```

202 // OPMODE 5
203 generate
204     if (CARRYINSEL == "OPMODE5") begin
205         assign carry_cascade = OPMODE[5];
206     end
207     else if (CARRYINSEL == "CARRYIN") begin
208         assign carry_cascade = CARRYIN;
209     end
210     else begin
211         assign carry_cascade = 1'b0;
212     end
213 endgenerate
214
215 // (9) instantiate CY1
216 REGISTER #(.WIDTH(1),
217             .RSSTYPE(RSSTYPE),
218             .REG(CARRYINREG))
219     u9 (.rst(RSTCARRYIN),
220         .CLK(CLK),
221         .en(CECARRYIN),
222         .in(carry_cascade),
223         .out(CIN));
224

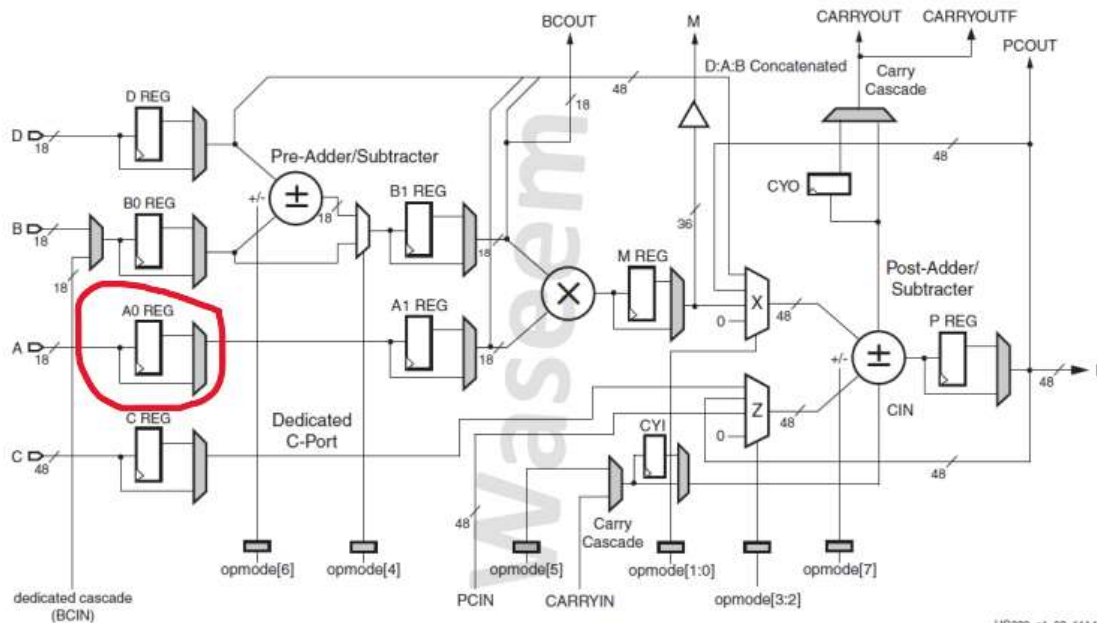
```

```

225 //----- S T A G E 4 -----
226
227 // ADDER OR SUBTRACT 2 OPMODE[7]
228 always @(*) begin
229     if (OPMODE[7] == 0)begin
230         out2_ADD_or_SUB [48:0] = Z + X + CIN ;
231     end
232     else begin
233         out2_ADD_or_SUB [48:0] = Z - X - CIN ;
234     end
235 end
236 assign CY0 = out2_ADD_or_SUB[48];
237 // (10) instantiate CY0
238 REGISTER #(.WIDTH(1),
239             .RSSTYPE(RSSTYPE),
240             .REG(CARRYOUTREG))
241     u10 (.rst(RSTCARRYIN),
242         .CLK(CLK),
243         .en(CECARRYIN),
244         .in(CY0),
245         .out(CARRYOUT));
246 assign CARRYOUTF = CARRYOUT ;
247
248 // (11) instantiate PREG
249 REGISTER #(.WIDTH(48),
250             .RSSTYPE(RSSTYPE),
251             .REG(PREG))
252     u11 (.rst(RSTP),
253         .CLK(CLK),
254         .en(CEP),
255         .in(out2_ADD_or_SUB[47:0]),
256         .out(P));
257
258 // PCOUT
259 assign PCOUT = P;
260
261 //-----DONE-----
262
263 endmodule //DSP48A1

```


2.2 Register Module



RTL Code

```

verilog > REGISTER.v > REGISTER
1  module REGISTER #(
2      parameter WIDTH =48,
3      parameter RSSTYPE ="SYNC",
4      parameter REG =1
5  )(
6      input CLK,rst,en,
7      input [WIDTH-1:0] in,
8      output reg [WIDTH-1:0] out
9  );
10 generate
11     if (REG == 1) begin
12         if (RSSTYPE == "SYNC")begin
13             always @(posedge CLK) begin
14                 if (rst == 1)
15                     out <= {WIDTH{1'b0}};
16                 else if(en)
17                     out <= in;
18                 end
19             end
20         else begin
21             always @(posedge CLK or posedge rst) begin
22                 if (rst == 1)
23                     out <= {WIDTH{1'b0}};
24                 else if(en)
25                     out <= in;
26                 end
27             end
28         end
29     else begin
30         always @(*) begin
31             out<=in;
32         end
33     end
34 endgenerate
35 endmodule //REGISTER

```

3. Testbench

RTL CODE

- Four different paths were tested to verify the design functionality.

```
verilog > TB.v > TB
1  module TB #(
2      // PARAMETERS
3      parameter A0REG = 0,
4      parameter A1REG = 1,
5      parameter B0REG = 0,
6      parameter B1REG = 1,
7      parameter CREG = 1,
8      parameter DREG = 1,
9      parameter MREG = 1,
10     parameter PREG = 1,
11     parameter CARRYINREG = 1,
12     parameter CARRYOUTREG = 1,
13     parameter OPMODEREG = 1,
14     parameter CARRYINSEL = "OPMODE5",
15     parameter B_INPUT = "DIRECT",
16     parameter RSSTYPE = "SYNC"
17 );
18 // MODULE REGISTER
19 reg rst;
20 reg CLK;
21 reg en;
22 // MODULE DSP48A1
23 reg [17:0] A,B,D;
24 reg [47:0] C;
25 reg CARRYIN;
26 reg [7:0] OPMODE;
27 reg [17:0] BCIN;
28 reg [47:0] PCIN;
29 wire [17:0] BCOUT;
30 wire [47:0] PCOUT;
31 wire [47:0] P;
32 wire [35:0] M;
33 wire CARRYOUT;
34 wire CARRYOUTF;
35 // EXPECTED OUTPUTS
36 reg [17:0] expected_BCOUT;
37 reg [47:0] expected_PCOUT;
38 reg [47:0] expected_P;
39 reg [35:0] expected_M;
40 reg expected_CARRYOUT;
41 reg expected_CARRYOUTF;
```

```

44 // instantiation DSP48A1
45 DSP48A1 #(A0REG,A1REG,B0REG,B1REG,CREG,DREG,MREG,PREG,CARRYINREG,CARRYOUTREG,OPMODEREG,CARRYINSEL,B_INPUT,RSSTYPE)
46 uut (.A(A), .B(B), .C(C), .D(D),
47     .CARRYIN(CARRYIN), .OPMODE(OPMODE),
48     .BCIN(BCIN), .PCIN(PCIN), .CLK(CLK),
49     .RSTA(rst), .RSTB(rst), .RSTC(rst), .RSTD(rst),
50     .RSTM(rst), .RSTP(rst), .RSTCARRYIN(rst), .RSTOPMODE(rst),
51     .CEA(en), .CEB(en), .CEC(en), .CED(en),
52     .CEM(en), .CEP(en), .CECARRYIN(en), .CEOPMODE(en),
53     .BCOUT(BCOUT), .PCOUT(PCOUT), .P(P), .M(M),
54     .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF)
55 );
56
57 initial begin
58     CLK=0;
59     forever
60         #1 CLK=~CLK;
61 end
62
63 //VERIFY RESET
64 integer i;
65 initial begin
66     rst = 1;
67     en = 0;
68     for (i = 0; i<15 ;i=i+1 ) begin
69         A=$random;
70         B=$random;
71         D=$random;
72         C=$random;
73         CARRYIN=$random;
74         OPMODE=$random;
75         BCIN=$random;
76         PCIN=$random;
77         @(negedge CLK);
78         if(BCOUT ==0 && PCOUT==0 && P==0 && M==0 && CARRYOUT==0 && CARRYOUTF==0)
79             $display("PASS");
80     end
81
82 // 2.2 Verify DSP Path 1
83 // delay for bug
84 rst = 0;
85
86 A=0;
87 B=0;
88 C= 0;
89 D=0;
90 CARRYIN=0;
91 en = 0;
92 OPMODE=0;
93 BCIN=0;
94 PCIN=0;
95 repeat(4) @(negedge CLK);
96 rst = 0;
97 en = 1;
98 @(negedge CLK);
99 A = 20; B = 10; C = 350; D = 25; OPMODE = 8'b11011101;
100 for (i = 0; i<20 ;i=i+1 ) begin
101     CARRYIN=$random;
102     BCIN=$random;
103     PCIN=$random;
104     repeat(4) @(negedge CLK);
105     expected_BCOUT = 18'hf;
106     expected_PCOUT = 48'h32;
107     expected_M = 36'h12c;
108     expected_P = 48'h32;
109     expected_CARRYOUT = 1'b0;
110     expected_CARRYOUTF = 1'b0;
111     // self cheking
112     if (BCOUT==expected_BCOUT &&
113         PCOUT == expected_PCOUT &&
114         M == expected_M &&
115         P == expected_P &&
116         CARRYOUT == expected_CARRYOUT &&
117         CARRYOUTF == expected_CARRYOUTF)
118         $display ("PASS");
119     else
120         $display ("FAIL");
121 end
122

```

```

123 // 2.3. Verify DSP Path 2
124 A = 20; B = 10; C = 350; D = 25; OPMODE = 8'b00010000;
125     for (i = 0; i<20 ;i=i+1 ) begin
126         CARRYIN=$random;
127         BCIN=$random;
128         PCIN=$random;
129         repeat(3) @(negedge CLK);
130         expected_BCOUT    = 18'h23;
131         expected_PCOUT    = 48'h0;
132         expected_M        = 36'h2bc;
133         expected_P        = 48'h0;
134         expected_CARRYOUT = 1'b0;
135         expected_CARRYOUTF = 1'b0;
136         // self cheking
137         if (BCOUT==expected_BCOUT &&
138             PCOUT == expected_PCOUT &&
139             M == expected_M &&
140             P == expected_P &&
141             CARRYOUT == expected_CARRYOUT &&
142             CARRYOUTF == expected_CARRYOUTF)
143             $display ("PASS");
144         else
145             $display ("FAIL");
146         end
147

```

```

148 // 2.4. Verify DSP Path 3
149 A = 20; B = 10; C = 350; D = 25; OPMODE = 8'b00001010;
150     for (i = 0; i<20 ;i=i+1 ) begin
151         CARRYIN=$random;
152         BCIN=$random;
153         PCIN=$random;
154         repeat(3) @(negedge CLK);
155         expected_BCOUT    = 18'ha;
156         expected_PCOUT    = 48'h0;
157         expected_M        = 36'hc8;
158         expected_P        = 48'h0;
159         expected_CARRYOUT = 1'b0;
160         expected_CARRYOUTF = 1'b0;
161         // self cheking
162         if (BCOUT==expected_BCOUT &&
163             PCOUT == expected_PCOUT &&
164             M == expected_M &&
165             P == expected_P &&
166             CARRYOUT == expected_CARRYOUT &&
167             CARRYOUTF == expected_CARRYOUTF)
168             $display ("PASS");
169         else
170             $display ("FAIL");
171         end
172

```

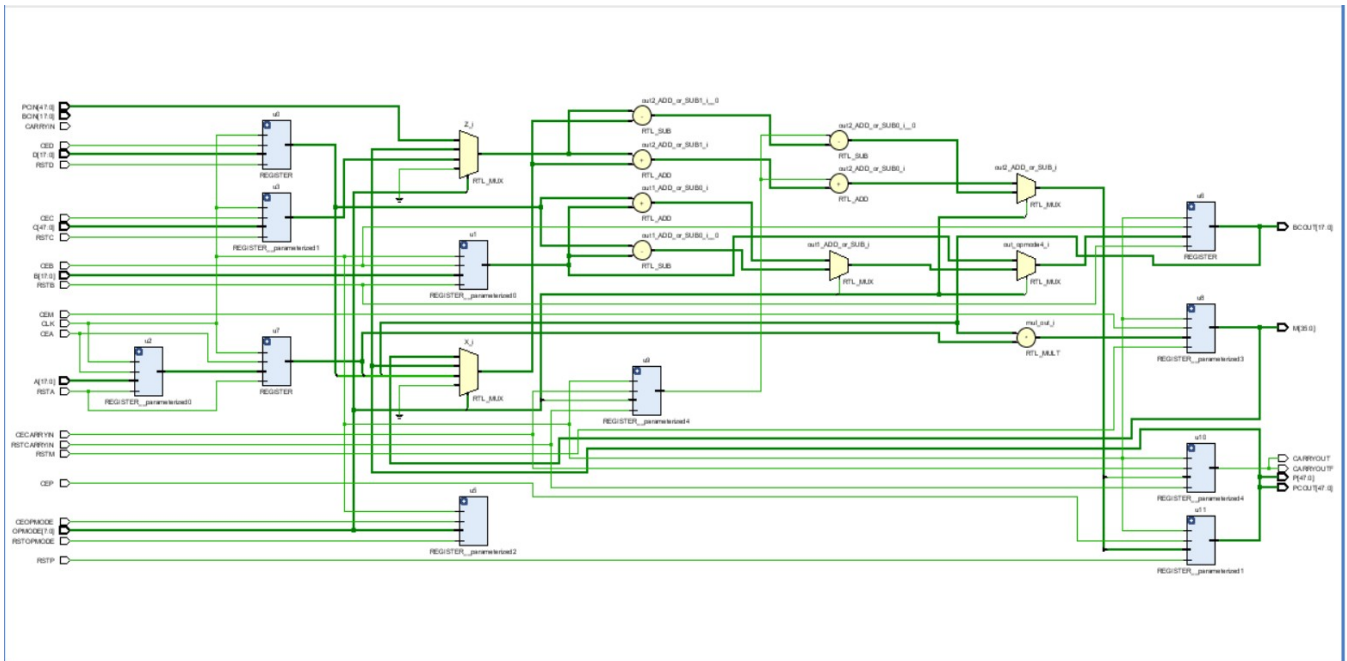


```

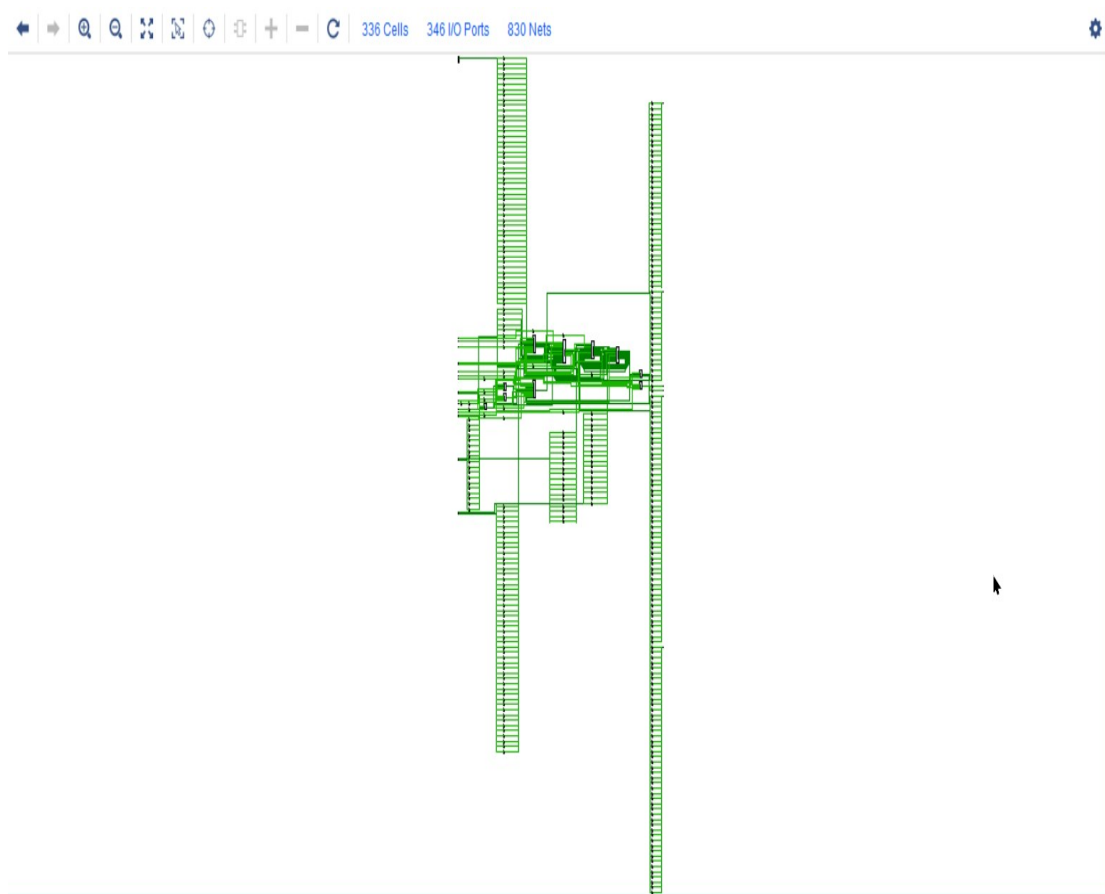
173 // 2.5. Verify DSP Path 4
174 A = 5; B = 6; C = 350; D = 25; OPMODE = 8'b10100111; PCIN = 3000;
175 for (i = 0; i<20 ;i=i+1 ) begin
176     CARRYIN=$random;
177     BCIN=$random;
178     repeat(3) @(negedge CLK);
179     expected_BCOUNT = 18'h6;
180     expected_PCOUNT = 48'hfe6fffec0bb1;
181     expected_M = 36'h1e;
182     expected_P = 48'hfe6fffec0bb1;
183     expected_CARRYOUT = 1'b1;
184     expected_CARRYOUTF = 1'b1;
185     // self cheking
186     if (BCOUT==expected_BCOUNT &&
187         PCOUT == expected_PCOUNT &&
188         M == expected_M &&
189         P == expected_P &&
190         CARRYOUT == expected_CARRYOUT &&
191         CARRYOUTF == expected_CARRYOUTF)
192         $display ("PASS");
193     else
194         $display ("FAIL");
195     end
196 #10;
197 $stop;
198 end
199 initial begin
200     $monitor(" A=%d B=%d C=%d D=%d OPMODE=%b | BCOUT=%h PCOUT=%h P=%h M=%h CARRYOUT=%b CARRYOUTF=%b",
201         A, B, C, D, OPMODE,
202         BCOUT, PCOUT, P, M, CARRYOUT, CARRYOUTF);
203 end
204 endmodule //TB

```

3. Elaboration Design



4. Synthesis

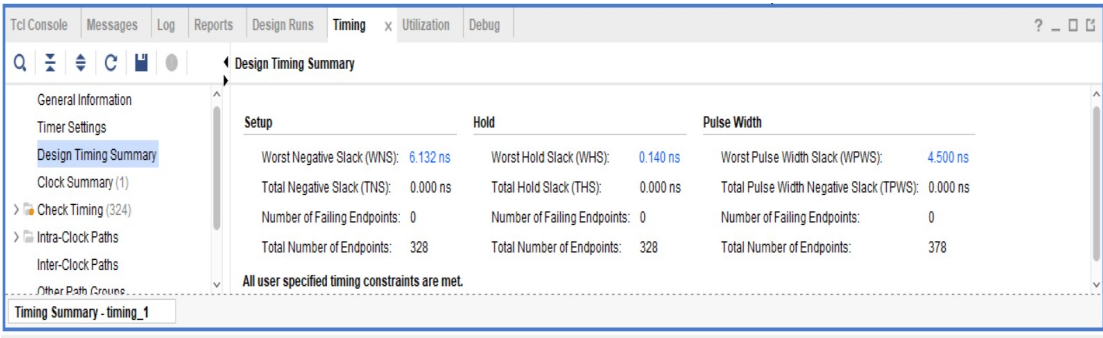


5. Utilization Report

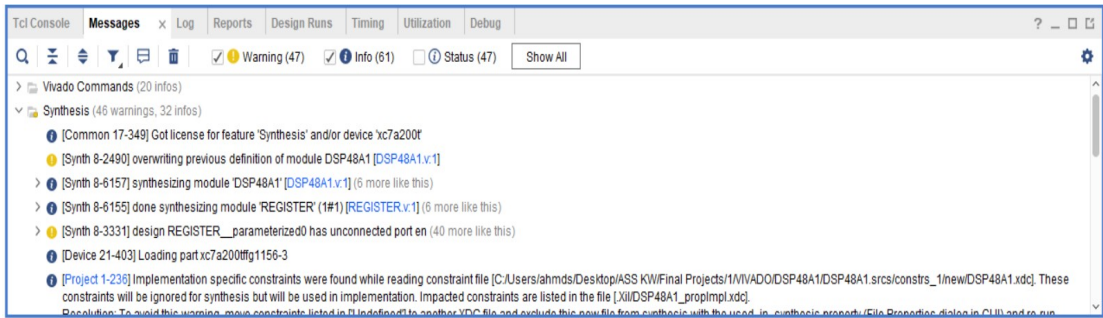
Tcl Console	Messages	Log	Reports	Design Runs	Utilization	Debug
Hierarchy						
Hierarchy Summary ▼ Slice Logic ▼ Slice LUTs (<1%) LUT as Logic (<1%) ▼ Slice Registers (<1%) Register as Flip Flop (▼)						
Name	Slice LUTs (134600)	Slice Registers (269200)	DSP s (740)	Bonded IOB (500)	BUFGCTRL (32)	
▼ DSP48A1	275	376	1	325	1	
u0 (REGISTER)	12	36	0	0	0	
u1 (REGISTER__para...	36	18	0	0	0	
u2 (REGISTER__para...	0	18	0	0	0	

utilization_1

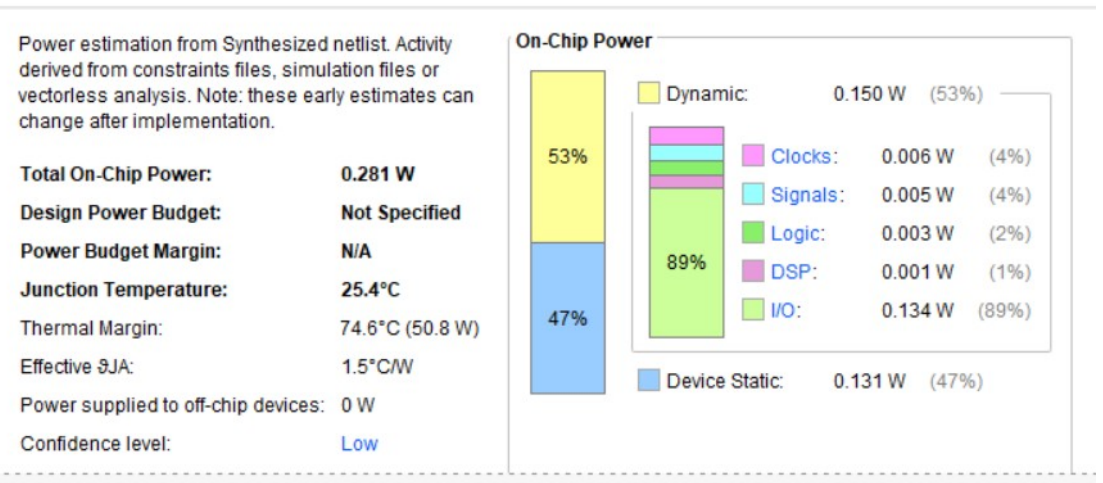
6. Timing Report



7. Message & Power



No Critical Warning



8. Do File

```
vlib work
vlog DSP48A1.v REGISTER.v TB.v
vsim -voptargs=+acc work.TB
add wave *
run -all
#quit -sim
```

9. Linting Tool

The screenshot displays the Lint Summary window of a design tool. The window is divided into two main sections: a table of linting errors and a summary table.

Linting Errors Table:

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
Warning	Uninspected	parameter_name_duplicate		Same parameter name is used in more than one mod...	DSP48A1	Nomenclature...	open	unassig...	1.1.4.3; 3.2.2.2
Warning	Uninspected	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassig...	3.5.6.4; 3.5.6.6
Warning	Uninspected	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassig...	3.5.6.4; 3.5.6.6
Warning	Uninspected	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassig...	3.5.6.4; 3.5.6.6
Warning	Uninspected	multi_ports_in_single_line		Multiple ports are declared in one line. Module REGIS...	REGISTER	Rtl Design Style	open	unassig...	3.5.6.3
Warning	Uninspected	multi_ports_in_single_line		Multiple ports are declared in one line. Module DSP48...	DSP48A1	Rtl Design Style	open	unassig...	3.5.6.3

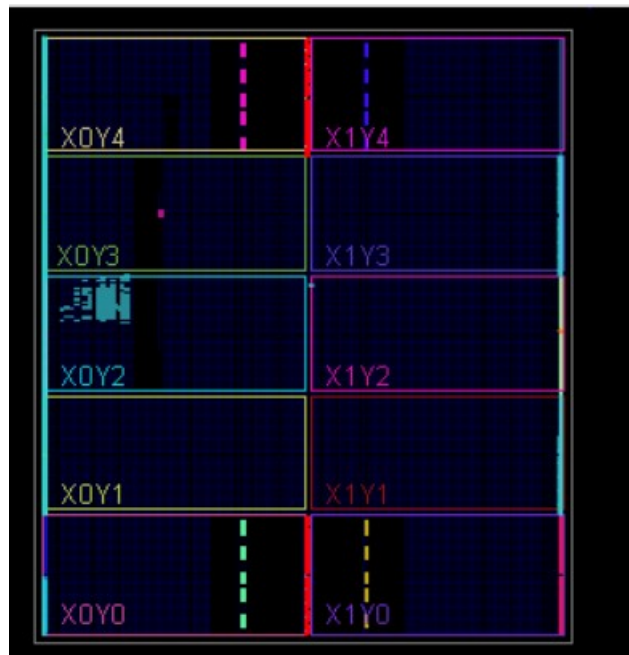
Summary Table:

Name	Count
Open(uninspected, pendi...	6
Info	6

The bottom status bar of the window shows: **Filter: Type here** | **Waived** | **Fixed** | **Pending** | **Uninspected** | **Bug** | **Verified** | **Total : 6 Selected : 0**

No ERROR

10. Implementaion



11. Simulation & Result

4 PATHES

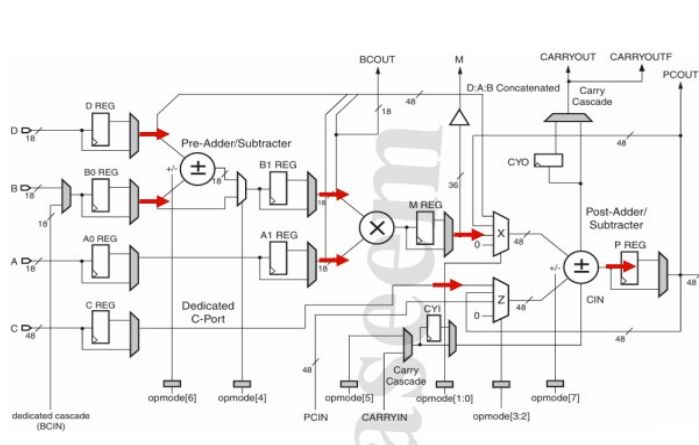


Figure 1: Path 1 Data Flow

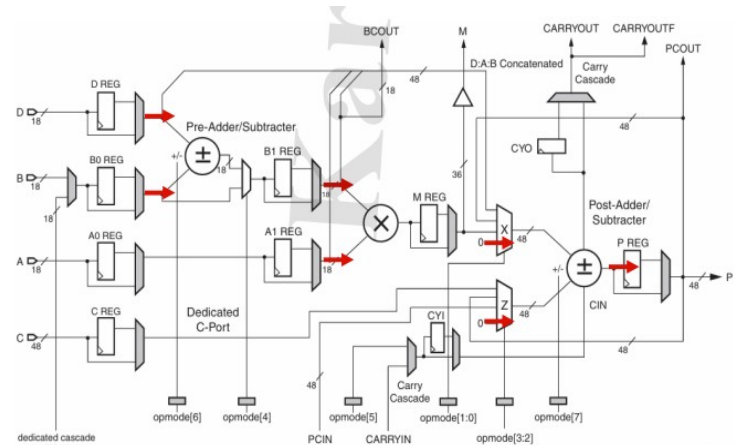


Figure 2: Path 2 Data Flow

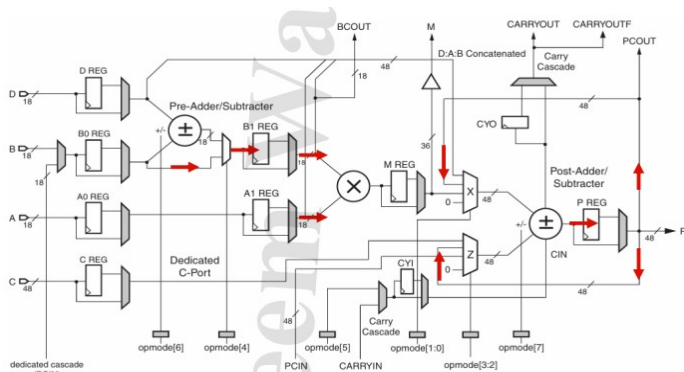


Figure 3: Path 3 Data Flow

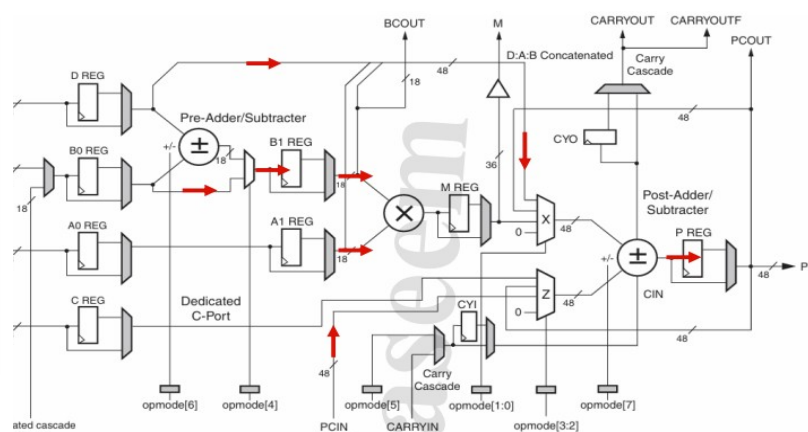
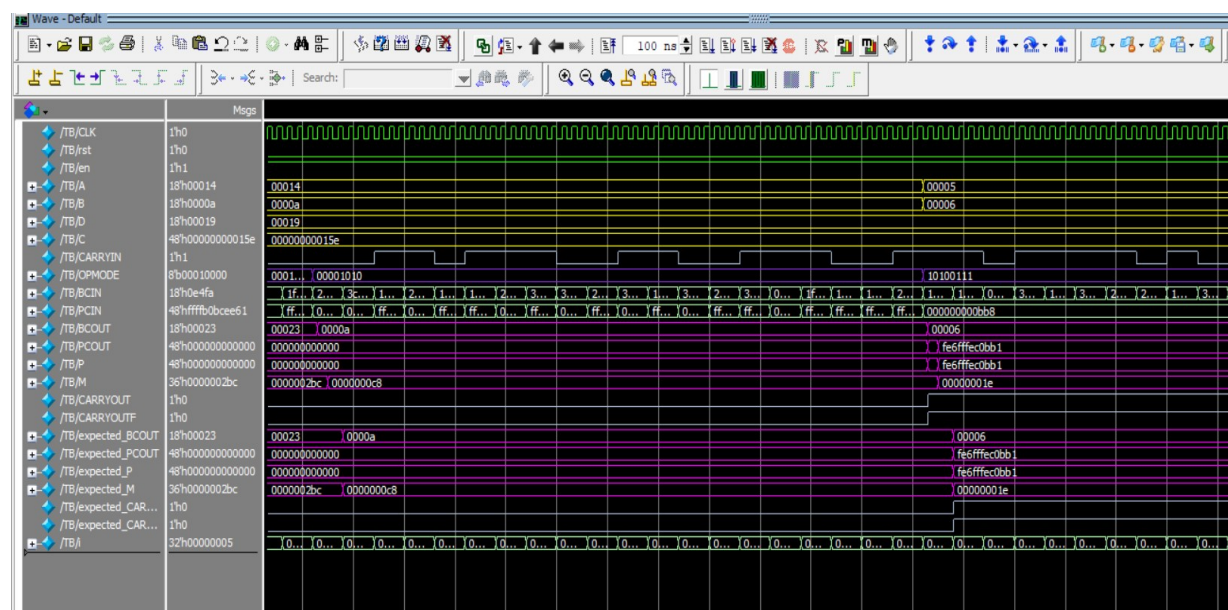
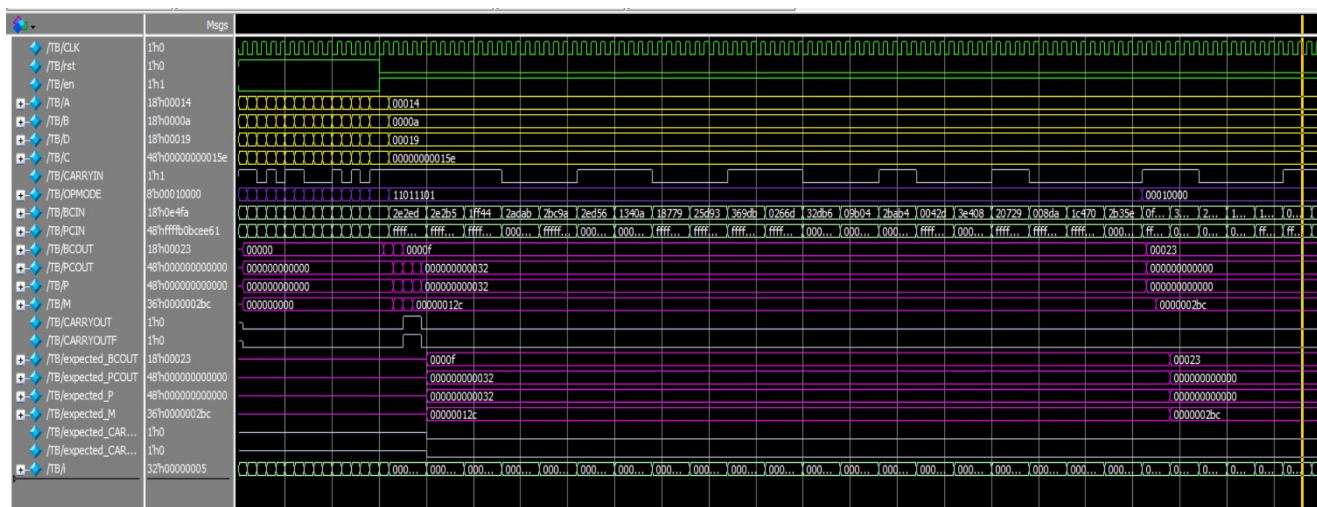


Figure 4: Path 4 Data Flow



transcript

Check reset

```

Transcript
Loading work.REGISTER(fast_5)
# A= 79140 B= 89729 C=281473667061347 D= 54793 OPMODE=10001101 | BCOUT=XXXXX PCOUT=XXXXXXXXXXXXX P=XXXXXXXXXXXXX M=XXXXXXXXXX CARRYOUT=x CARRYOUTF=x
# A= 79140 B= 89729 C=281473667061347 D= 54793 OPMODE=10001101 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=254721 B=249101 C= 512609597 D=258422 OPMODE=10001100 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=230597 B=250538 C=281473832874615 D=260069 OPMODE=10001111 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A= 31464 B=151237 C=281474415601853 D= 18780 OPMODE=01100101 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=205440 B= 8480 C=281474151271581 D= 17834 OPMODE=00010011 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=253291 B=142037 C=281474294496942 D=215554 OPMODE=11001111 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=199370 B= 85052 C= 1160667530 D= 48626 OPMODE=11011000 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A= 69099 B=157110 C= 1460999086 D=260550 OPMODE=010101010 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=147845 B=152911 C= 486945594 D= 24635 OPMODE=00010101 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=198498 B=129868 C= 1203347855 D= 87455 OPMODE=10110111 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A=245851 B=145289 C=281474576367312 D=143945 OPMODE=01010001 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# PASS
# A= 52930 B=257480 C= 519236925 D=154231 OPMODE=011111110 | BCOUT=00000 PCOUT=0000000000000000 F=0000000000000000 M=0000000000 CARRYOUT=0 CARRYOUTF=0
# ---

```

Check path 1

```
# A= 20 B= 10 C= 350 D= 25 OPMODE=00001010 | BCOUT=00023 PCOUT=000000000000 P=000000000000 M=0000002bc CARRYOUT=0 CARRYOUTF=0
# A= 20 B= 10 C= 350 D= 25 OPMODE=00001010 | BCOUT=0000a PCOUT=000000000000 P=000000000000 M=0000002bc CARRYOUT=0 CARRYOUTF=0
# A= 20 B= 10 C= 350 D= 25 OPMODE=00001010 | BCOUT=0000a PCOUT=000000000000 P=000000000000 M=0000000c8 CARRYOUT=0 CARRYOUTF=0
# PASS
# PASS
# PASS
```

Check path 2

```
# A= 20 B= 10 C= 350 D= 25 OPMODE=00010000 | BCOUT=0000f PCOUT=000000000032 P=000000000032 M=00000012c CARRYOUT=0 CARRYOUTF=0
# A= 20 B= 10 C= 350 D= 25 OPMODE=00010000 | BCOUT=00023 PCOUT=000000000000 P=000000000000 M=00000012c CARRYOUT=0 CARRYOUTF=0
# A= 20 B= 10 C= 350 D= 25 OPMODE=00010000 | BCOUT=00023 PCOUT=000000000000 P=000000000000 M=0000002bc CARRYOUT=0 CARRYOUTF=0
# PASS
# PASS
# PASS
```

Check path 3

```
# A= 5 B= 6 C= 350 D= 25 OPMODE=10100111 | BCOUT=0000a PCOUT=000000000000 P=000000000000 M=0000000c8 CARRYOUT=0 CARRYOUTF=0
# A= 5 B= 6 C= 350 D= 25 OPMODE=10100111 | BCOUT=00006 PCOUT=fe6ffffb00bae P=fe6ffffb00bae M=0000000c8 CARRYOUT=1 CARRYOUTF=1
# A= 5 B= 6 C= 350 D= 25 OPMODE=10100111 | BCOUT=00006 PCOUT=fe6ffffec0bb1 P=fe6ffffec0bb1 M=00000001e CARRYOUT=1 CARRYOUTF=1
# PASS
# PASS
# PASS
```

Check path 4

```
# A= 5 B= 6 C= 350 D= 25 OPMODE=10100111 | BCOUT=00006 PCOUT=fe6ffffb00bae P=fe6ffffb00bae M=0000000c8 CARRYOUT=1 CARRYOUTF=1
# A= 5 B= 6 C= 350 D= 25 OPMODE=10100111 | BCOUT=00006 PCOUT=fe6ffffec0bb1 P=fe6ffffec0bb1 M=00000001e CARRYOUT=1 CARRYOUTF=1
# PASS
# PASS
# PASS
```


11. FPGA

SPARTAN 6

